

Manual de Instalación y Producción

Sistema: RutaSocial

Este manual cubre la instalación local para desarrollo, el despliegue en producción provisional (Railway) y la migración a producción real con dominios propios del cliente.

Índice

1. [Requisitos](#)
2. [Variables de Entorno](#)
3. [Instalación Local con Docker Compose](#)
4. [Desarrollo sin Docker](#)
5. [Verificación](#)
6. [Producción Provisional \(Railway\)](#)
7. [Producción Real \(Dominios Propios\)](#)
8. [MongoDB Atlas](#)
9. [Solución de Problemas](#)

1. Requisitos

- **Git** para clonar el repositorio y control de versiones.
- **Docker Desktop** con Docker Compose (para instalación local rápida).
- **Node.js** (versión 23.x o compatible) para desarrollo frontend.
- **Go 1.24** para desarrollo y compilación backend.
- **Cuenta en MongoDB Atlas** (gratuita) para la base de datos en producción.
- **Cuenta en Railway o servidor VPS** para el despliegue.

2. Variables de Entorno

Backend (`Backend/.env`)

Copiar desde `Backend/.env.example` :

```
MONGODB_URI=mongodb+srv://usuario:contraseña@cluster.mongodb.net/?appName=app
MONGODB_DB_NAME=rutasocial
JWT_SECRET=clave_secreta_segura_de_al_menos_32_caracteres
ADMIN_EMAIL=admin@ejemplo.com
```

```
ADMIN_PASSWORD=Admin12345
FRONTEND_URL=https://tudominio.com
FRONTEND_URL_MOBILE=null
APP_ENV=development
EMAIL_FROM=noreply@ejemplo.com
EMAIL_PASS=contraseña_correo
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
```

Variable	Obligatorio	Descripción
MONGODB_URI	Sí	URI de conexión a MongoDB Atlas. Para desarrollo local con contenedor Docker usar <code>mongodb://mongo:27017</code> .
MONGODB_DB_NAME	Sí	Nombre de la base de datos en MongoDB.
JWT_SECRET	Sí	Clave para firmar tokens JWT. Debe ser segura y única en producción (mínimo 32 caracteres).
ADMIN_EMAIL	Sí	Correo del administrador inicial que se crea al iniciar el sistema por primera vez.
ADMIN_PASSWORD	Sí	Contraseña del administrador inicial.
FRONTEND_URL	Sí	URL del frontend web para CORS. En producción, el dominio real (<code>https://tudominio.com</code>).
FRONTEND_URL_MOBILE	No	URL del frontend móvil para CORS. Usar <code>null</code> si no se usa.
APP_ENV	Sí	<code>development</code> o <code>production</code> . En production oculta Swagger y aplica validaciones extra.
EMAIL_FROM	No	Dirección remitente para correos del sistema.
EMAIL_PASS	No	Contraseña de aplicación del correo SMTP.
SMTP_HOST	No	Servidor SMTP (ej: <code>smtp.gmail.com</code>).
SMTP_PORT	No	Puerto SMTP (ej: <code>587</code> para TLS).

FrontendWeb (`FrontendWeb/.env`)

Copiar desde `FrontendWeb/.env.example` :

```
VITE_URL_BACKEND=http://localhost:8080
VITE_BASE_URL=
```

Variable	Obligatorio	Descripción
<code>VITE_URL_BACKEND</code>	Sí	URL completa del backend. En local: <code>http://localhost:8080</code> . En producción: <code>https://api.tudominio.com</code> .
<code>VITE_BASE_URL</code>	No	Ruta base para la aplicación (vacío para raíz, o <code>/ruta</code> si se sirve desde un subdirectorio).

3. Instalación Local con Docker Compose

Ideal para desarrollo y pruebas rápidas. Requiere Docker Desktop.

1. Abrir terminal en la raíz del repositorio.
2. Crear los archivos `.env` a partir de los `.env.example` .
3. Construir las imágenes:

```
docker compose build
```

4. Levantar los servicios:

```
docker compose up -d
docker compose ps
```

5. Verificar que los contenedores estén saludables:

```
docker compose logs --tail=20 backend
docker compose logs --tail=20 frontend
```

En el entorno local, `docker-compose.yml` configura `APP_ENV=development` para que Swagger esté disponible. El Mongo del contenedor se usa automáticamente.

Comandos útiles

```
# Ver logs en tiempo real
docker compose logs -f backend
```

```
# Detener servicios (sin borrar datos)
docker compose down

# Detener y borrar volúmenes (se pierde la BD local)
docker compose down -v

# Reconstruir un servicio específico
docker compose build backend
docker compose up -d
```

4. Desarrollo sin Docker

Backend

```
cd Backend
go mod download
go test ./...
go run ./cmd/main.go
```

Requiere tener `Backend/.env` configurado y una instancia de MongoDB corriendo (local o Atlas).

FrontendWeb

```
cd FrontendWeb
npm ci
npm run typecheck
npm run test
npm run dev
```

El frontend en desarrollo corre en `http://localhost:5173` (o el puerto que asigne Vite).

FrontendMovil (legado)

El proyecto móvil con React Native + Expo se conserva como referencia pero no se usa activamente. La estrategia actual es generar APK desde `FrontendWeb`.

5. Verificación

Después de iniciar los servicios (locales o en producción), verificar:

- **Frontend:** `http://localhost:3000` (local) o el dominio configurado. Debe cargar la aplicación sin errores.

- **Backend health:** `http://localhost:8080/healthz` debe responder `{"status":"ok"}`.
- **Backend ready:** `http://localhost:8080/readyz` debe responder `{"status":"ready"}` (verifica conexión a MongoDB).
- **Swagger (solo development):** `http://localhost:8080/swagger/index.html`.
- **Tests:** Ejecutar `go test ./...` en Backend y `npm run test` en FrontendWeb.

6. Producción Provisional (Railway)

Esta sección explica cómo desplegar en Railway, que es el servicio usado durante el desarrollo y la fase provisional del proyecto.

Lo descrito en esta sección corresponde a la configuración actual del proyecto. Cuando el cliente migre a sus propios dominios, debe seguir la sección 7.

6.1 Preparar el Backend

1. Asegurarse de que `Backend/Dockerfile` compile y exponga el puerto correcto (el backend escucha en el puerto definido por `PORT` o `8080`).
2. El backend **no** debe incluir archivos `.env` en la imagen (ya están en `.dockerignore`).
3. En Railway, configurar las variables de entorno desde el panel (ver sección 2).

6.2 Preparar el Frontend

1. En `FrontendWeb/.env` (para el build) o como variable de entorno en Railway, configurar `VITE_URL_BACKEND` apuntando al backend desplegado (ej: `https://web-production-1234.up.railway.app`).
2. El `Dockerfile` del frontend valida que `VITE_URL_BACKEND` no sea `localhost` cuando `APP_ENV=production`. Si se configura correctamente, el build pasará.
3. Railway construye automáticamente desde el `Dockerfile`. Si se usa un servicio estático, copiar el contenido de `dist/`.

Railway asigna dominios aleatorios del tipo `*.up.railway.app`. Estos dominios se usan durante la fase provisional. Para producción real se reemplazan por dominios personalizados.

6.3 Configurar Railway

1. Crear cuenta en [Railway](#) (ofrece crédito gratuito inicial).
2. Iniciar sesión y conectar el repositorio de GitHub.
3. Crear dos servicios:

- **Backend:** Seleccionar `Backend/` como directorio raíz, tipo "Dockerfile".
 - **Frontend:** Seleccionar `FrontendWeb/` como directorio raíz, tipo "Dockerfile".
4. Agregar una base de datos MongoDB (Railway ofrece MongoDB como plugin) o usar MongoDB Atlas externo.
 5. Configurar las variables de entorno en cada servicio desde el panel de Railway.

6.4 Variables en Railway (Backend)

```
MONGODB_URI=mongodb+srv://...  
MONGODB_DB_NAME=rutasocial  
JWT_SECRET=clave_segura_aleatoria  
ADMIN_EMAIL=admin@tudominio.com  
ADMIN_PASSWORD=Admin12345  
FRONTEND_URL=https://frontend.up.railway.app  
FRONTEND_URL_MOBILE=null  
APP_ENV=production
```

6.5 Despliegue Manual (CLI Railway)

```
npm install -g railway  
railway login  
railway link  
railway up
```

7. Producción Real (Dominios Propios del Cliente)

Cuando el cliente adquiera sus propios dominios, se deben seguir estos pasos para migrar desde la infraestructura provisional.

Antes de iniciar, el cliente debe tener:

- Un dominio para el frontend (ej: `https://rutasocial.cliente.cl`).
- Un dominio o subdominio para el backend (ej: `https://api.rutasocial.cliente.cl`).
- Acceso a un servicio de hosting (Railway, Render, VPS, AWS, etc.).
- Certificados SSL/TLS para ambos dominios.

7.1 Opciones de Hosting

Plataforma	Pros	Contras
Railway	Fácil, dominios personalizados, Docker nativo.	Costo variable según uso.
Render	Tiene capa gratuita, dominios personalizados.	Sin soporte Docker nativo en gratis.
VPS (DigitalOcean, AWS EC2, etc.)	Control total, escalable.	Requiere configuración manual de servidor web, SSL, etc.
Cloudflare Pages + Workers	Frontend estático rápido, SSL automático.	Backend necesita Workers o servidor externo.

7.2 Configuración de Dominios

DNS

Crear registros DNS apuntando a la infraestructura elegida:

```

tipo      nombre                destino
A         api.rutasocial.cliente.cl  <IP del servidor>
CNAME     rutasocial.cliente.cl      <dominio de Railway/Render>

```

Backend: Configurar CORS

En las variables de entorno del backend, `FRONTEND_URL` debe ser el dominio real del frontend:

```
FRONTEND_URL=https://rutasocial.cliente.cl
```

Si hay múltiples orígenes (web + app móvil), el backend debe modificarse para aceptar una lista.

Frontend: Apuntar al Backend

En las variables de entorno del frontend (en tiempo de build):

```
VITE_URL_BACKEND=https://api.rutasocial.cliente.cl
```

Luego reconstruir y redeployar el frontend.

7.3 Seguridad

- **JWT_SECRET:** Generar una clave única y segura (ej: `openssl rand -base64 32`).
- **HTTPS:** Usar SSL/TLS en todos los dominios. Railway y Render proveen SSL automático con dominios personalizados. En VPS, usar Let's Encrypt con Certbot.
- **MongoDB:** Restringir acceso por IP en Atlas. Usar contraseñas fuertes y, de ser posible, autenticación por certificado.
- **Encryptado en tránsito:** Asegurar que `MONGODB_URI` incluya `ssl=true` o `tls=true` .
- **Rate limiting:** El backend ya incluye rate limiting en login y register (5 peticiones/minuto). Verificar que esté activo en producción.

7.4 Migración de Datos

Si ya hay datos en la base de datos provisional:

1. Exportar la base de datos desde MongoDB Atlas o Railway:

```
mongodump --uri="mongodb+srv://..." --out=./backup
```

2. Importar en la nueva base de datos:

```
mongorestore --uri="mongodb+srv://..." --dir=./backup
```

3. Actualizar las variables de entorno para que apunten a la nueva base de datos.

No compartir la misma base de datos entre desarrollo y producción. Usar bases de datos separadas.

7.5 Despliegue Manual en VPS

Si se opta por un VPS (ej: DigitalOcean, AWS EC2):

1. Conectarse por SSH al servidor.
2. Instalar Docker y Docker Compose.
3. Clonar el repositorio en el servidor.
4. Crear los archivos `.env` con valores de producción.
5. Construir y levantar:


```
docker compose -f docker-compose.prod.yml build
docker compose -f docker-compose.prod.yml up -d
```

Para producción real se recomienda crear un archivo `docker-compose.prod.yml` separado que no incluya MongoDB (usar Atlas externo) y que configure correctamente los dominios y puertos.

7.6 Despliegue con Imágenes en Docker Hub / Registry

Para un flujo más profesional, subir las imágenes a un registry y luego desplegarlas:

```
docker tag rutasocial-backend usuario/backend:latest
docker tag rutasocial-frontend usuario/frontend:latest
docker push usuario/backend:latest
docker push usuario/frontend:latest
```

Luego en el servidor:

```
docker pull usuario/backend:latest
docker pull usuario/frontend:latest
docker compose up -d
```

8. MongoDB Atlas

Se recomienda MongoDB Atlas para producción por su facilidad de uso y capa gratuita (512 MB).

1. Crear cuenta en [MongoDB Atlas](#).
2. Crear un cluster gratuito (M0).
3. En "Network Access", agregar la IP del servidor de producción (o `0.0.0.0/0` si se usa autenticación fuerte).
4. En "Database Access", crear un usuario con contraseña segura.
5. Obtener la URI de conexión y configurarla como `MONGODB_URI`.
6. Al iniciar el backend por primera vez, se crearán índices automáticamente (verificar en logs).

9. Solución de Problemas

CORS bloqueado

- Verificar que `FRONTEND_URL` en `Backend/.env` coincida exactamente con el dominio del frontend (incluyendo protocolo `https://`).

- Reiniciar el backend después de cambiar esta variable.

MongoDB no conecta

- Verificar `MONGODB_URI` .
- En Atlas, verificar que la IP esté whitelisteada en Network Access.
- Verificar que el usuario de la base de datos tenga permisos de lectura/escritura.

Error 409 al crear ruta

- El nombre de la ruta ya existe en el sistema. Usar un nombre diferente.
- La validación es sin distinción de mayúsculas/minúsculas y espacios al inicio/final.

Error 403 al iniciar sesión

- La cuenta está pendiente de aprobación. Contactar a un administrador.

El frontend muestra pantalla en blanco o errores de carga

- Verificar que `VITE_URL_BACKEND` esté correcto y el backend esté accesible.
- Limpiar caché del navegador (Ctrl+F5).
- Revisar la consola del navegador para errores específicos.

Error "dist/index.html not found" en build Docker

- El build de Vite falló. Revisar errores previos en la salida del build.
- Verificar que `npm run build` funcione localmente.

Puerto ocupado

- Si el puerto 3000 o 8080 está ocupado, cambiar los puertos en `docker-compose.yml` .
- Para desarrollo frontend local, Vite asigna automáticamente el siguiente puerto disponible.

Para reportar errores o solicitar ayuda técnica, contactar al equipo de desarrollo.
No eliminar volúmenes Docker si se desea conservar la base de datos local.